

SOFTWARE REQUIREMENTS SPECIFICATION

Personal & Household Budget Tracking Application

Version: 1.0.0

Date: June 20, 2026

Status: Approved Specification

Author: Antigravity Pairing Partner

Software Requirements Specification (SRS)

Personal & Household Budget Tracking Application

Version: 1.0.0
Date: June 20, 2026
Status: Approved Specification
Author: AntigraVity (Advanced Agentic Coding Partner)

1. Project Overview

1.1 Goal

The goal of the Personal & Household Budget Tracking Application is to empower individuals, couples, and families to manage their personal finances through collaborative tracking of income, expenses, budgets, savings goals, loans, investments, and net worth. The system is designed with an India-First focus, optimizing for mobile-first standalone browser usage, offering a seamless user experience that enables recording an expense in less than 5 seconds.

1.2 Problem Solved

Traditional budgeting tools are either overly complex, lack real-time collaborative features for household members, or fail to provide actionable financial insights beyond recording raw transactions. This application addresses:

- Friction in entry: Slow and cumbersome transaction entry screens.
- Lack of collaboration: Inability to manage household-wide budgets with multiple family members under distinct roles.
- Actionable Narrative: Standard dashboard graphs fail to tell a cohesive financial "story." The Monthly Review report aggregates data into a narrative that guides user behavior.

1.3 Key Value Propositions

- Sub-5 Second Expense Entry: A 3-tap optimized mobile workflow utilizing custom in-app keypads, favourite categories, and recent history.
- Structured Household Collaboration: A multi-user household system using Role-Based Access Control (RBAC) to separate viewers, members, and owners.
- Hybrid Connectivity: A semi-offline local drafts queue built on `localStorage` to buffer entries when network connection is dropped (common in basement merchant shops in India).
- Multi-Horizon Visualization: Instantly switch budget views between Weekly (pro-rated day-by-day), Monthly (primary calendar month), and Yearly (aggregates with rolling projections).

2. Overall Description

2.1 User Personas

Persona 1: Arjun -- The Single Working Professional

- Role: Single user, primary earner.
- Tech Savvy: High.
- Goals: Fast daily expense tracking, automated recurring bill alerts, monthly savings target visualization.
- Pain Points: Forgets to log micro-transactions; wants to budget Swiggy, Zomato, and Uber expenses.

Persona 2: The Mehta Family -- Collaborative Household

- Role: Multiple users (Parents + 2 Teens).
- Tech Savvy: Low to Medium.
- Goals: Cross-member spending visibility, allowance tracking, family vacation goal progress.
- Pain Points: Lack of visibility into children's spend; cash transactions mixed with digital UPI payments.

Persona 3: Kavya -- The Freelancer

- Role: Single user with variable income.
- Tech Savvy: High.
- Goals: Track irregular client payments, calculate quarterly advance tax estimates, separate business from personal expenses.
- Pain Points: Irregular cash flow makes fixed monthly budgeting difficult.

2.2 Assumptions & Constraints

- Solo Developer: The application must be built and maintained by a single engineer working weekends (~16 hours/week). Boilerplate and operational overhead must be minimized.
- 90% Mobile-First: Responsive viewports between 320px and 480px are the primary interfaces. Native gestures (swipes, pull-to-refresh) must feel fluid.
- Server-First MVP: Complex background workers, IndexedDB sync engines, and conflict resolution are deferred to Phase 2. Data is committed directly to the remote server. Offline states are handled via a local draft queue.
- India-First: Currency defaults to Indian Rupee (INR - Rs.), dates default to DD/MM/YYYY, numbers default to Indian lakhs/crores formatting (e.g., Rs.1,23,456.00), and default categories align with Swiggy, Zomato, Ola, and Paytm services.

3. Functional Requirements (FR)**3.1 Authentication & Onboarding**

- FR-001: Email/Password Registration: The system must allow users to register using an email address and a strong password (minimum 8 characters, 1 number, 1 special character).
- FR-002: Google OAuth Login: Users must be able to log in or register via Google OAuth. Registration must automatically populate their profile, timezone, and avatar.
- FR-003: Two-Factor Authentication (2FA): Users must be able to enable TOTP-based 2FA in their settings. High-security actions (e.g., inviting household members, exporting JSON data) must prompt for a 2FA code if enabled.

3.2 Household & Collaborators

- FR-004: Multi-User Households: Users must be able to create or join a household. A user can belong to up to 3

households in the Premium tier (1 in Free tier).

- FR-005: Role-Based Access Control (RBAC): The system must enforce roles:
 - Owner: Full CRUD on all data, invite/remove members, delete household.
 - Co-Owner: Full CRUD on transactions, budgets, goals, and bills; cannot delete the household or remove the Owner.
 - Member: Create, read, update, delete own transactions; read-only access to household budgets and goals.
 - Viewer: Read-only access to overall reports and summaries; cannot view individual transaction notes or add records.

3.3 Transaction Management

- FR-006: Transaction Entry: The system must allow recording transactions categorized as Income, Expense, or Transfer. Required fields: Amount, Category, Date, Account. Optional fields: Description, Notes, Tags, Reimbursable flag.
- FR-007: Split Transactions: Users must be able to split a single transaction across multiple categories. The system must validate that the sum of the splits exactly matches the transaction total.
- FR-008: Recurring Transactions: The system must support generating recurring transactions on set intervals (daily, weekly, bi-weekly, monthly, quarterly, yearly). The scheduling engine (BullMQ) must compute and insert these on the designated date.

3.4 Budgets & Visualization

- FR-009: Category Budgets: Users must be able to set monthly planned budget limits per category (or subcategory). Subcategory budgets take precedence over parent budgets.
- FR-010: Period Switcher: The interface must support switching the budget view between:
 - Weekly: Aggregated daily. Pro-rates monthly budgets to a weekly scale ($\text{Monthly Budget} \div 4.33$).
 - Monthly: Primary budget planning. Shows current month progress, remaining cash, and rollover credits.
 - Yearly: Multi-month bar chart displaying calendar year or Indian Financial Year (April to March) with 3-month rolling averages projecting future months.
- FR-011: Budget Rollovers: The system must support rolling over unspent budget amounts to the subsequent month. This must be configurable per category.

3.5 Savings Goals & Bills

- FR-012: Savings Goals: Users must be able to create savings goals with target amounts, target dates, and linked bank accounts. Goal contributions must be recorded, tracking progress towards completion.
- FR-013: Bill Tracking & Reminders: The system must allow users to log upcoming bills (due day, fixed or variable amount). The system must trigger alerts N days before the due date.

3.6 Reports & Narrative

- FR-014: Monthly Narrative Review: The system must generate a textual financial narrative alongside charts at the end of each month (e.g., "Your Food & Dining spend was Rs.2,200 over budget due to 4 Swiggy deliveries in Week 2").
- FR-015: Interactive Charts: All reports must render high-DPI canvas charts (Chart.js) that allow users to tap segments to drill down into transaction lists.
- FR-016: Financial Health Score: The system must calculate a monthly health score (0-100) based on: budget adherence (40%), savings rate (30%), emergency fund coverage (20%), and debt-to-income ratio (10%).

4. Non-Functional Requirements (NFR)

4.1 Performance & Resource Limits

- NFR-001: Mobile Load Speed: The application shell must load and become interactive in < 3.0 seconds (TTI) on a throttled 3G/4G connection. First Contentful Paint (FCP) must be < 1.5 seconds.
- NFR-002: Sub-Second Save: Transaction saves must process and return a successful UI response in < 1.0 second under standard network conditions.
- NFR-003: JavaScript Payload: The initial critical JS bundle size must not exceed 80KB (gzipped). Non-critical dependencies (e.g., Chart.js) must be lazy-loaded.

4.2 Security & Compliance

- NFR-004: Encryption at Rest: Sensitive database columns (`totp\_secret`, `bank\_access\_token`) must be encrypted using AES-256-GCM at the application level.
- NFR-005: Session Security: Authentication must use short-lived JWT access tokens (15 minutes) and HttpOnly, Secure, SameSite=Strict cookies for Refresh Tokens.
- NFR-006: Audit Logging: All mutation actions (CREATE, UPDATE, DELETE) must write a row to the `audit\_log` table. No passwords or secrets must ever be logged.

4.3 Reliability & Accessibility

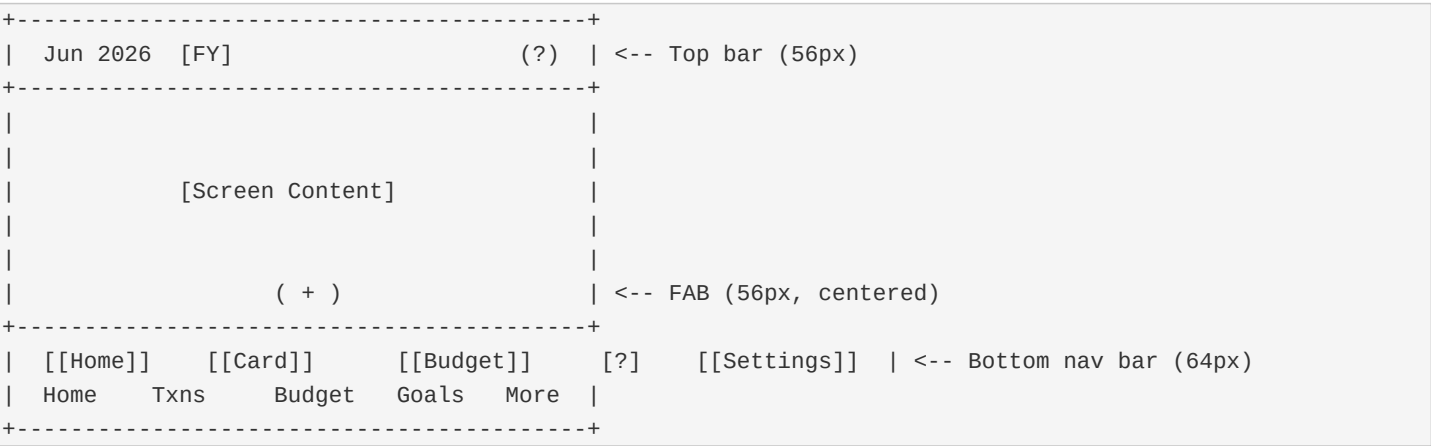
- NFR-007: Safe-Area Padding: The UI layout must respect hardware features such as camera notches, bottom bars, and safe zones using CSS `env(safe-area-inset-\*)`.
- NFR-008: Touch Targets: All interactive elements (buttons, links, tab items) must have a minimum touch target area of 44x44px with a minimum separation of 8px.

5. Screen Specifications & Mobile-First UX

5.1 Main Layout & Bottom Navigation Bar

The application enforces a global shell layout on screens <=768px:

- Bottom Tab Bar: Height is 64px + `env(safe-area-inset-bottom)`. It contains five navigation tabs: Home (Dashboard), Transactions, Budget, Goals, and More.
- Floating Action Button (FAB): A 56x56px primary circular button centered above the tab bar.
- Tap: Instantly opens the Quick-Add Expense sheet.
- Long-press: Displays a speed dial overlay to choose: [Add Expense] (default), [Add Income], or [Add Transfer].



5.2 Dashboard Screen

- Balance Card: Renders "Available Balance" (Total Assets - Total Liabilities) in a large font. Displays a progress bar representing overall monthly budget spent (e.g., "Rs.52,340 spent of Rs.85,000 | 62% used").
- Category Alerts: A card listing categories that have breached 80% or 100% of their budgeted limits.
- Quick Stats Grid: Mini metrics for Upcoming Bills (next 3 days) and Savings Goal Progress.
- Recent Activity: A list of the last 3 logged transactions. Each item supports swipe gestures: swipe left to reveal [Edit/Delete], swipe right to mark as [Reviewed].

5.3 Quick-Add Expense Sheet

A bottom sheet that animates upward (200ms transform) when the FAB is tapped. Designed to bypass the system keyboard to optimize entry speed.

- Keypad Interface: Renders an in-app grid numeric keypad. The current amount is displayed in a large, centered font.
- Step 2 (Categories): Once a non-zero amount is entered, a panel slides in from the right:
- Recent Row: 5 horizontal chips representing the user's most frequently selected categories.
- Pinned Row: Favourited categories (up to 5, marked with a star).
- List: An expandable tree list of all active categories.
- Step 3 (Collapsible Details): Optional details (Date, Account switcher, Notes, Tags, Receipt image upload via camera environment capture) are collapsed under a "More Options" toggle.
- Happy Path: Tap FAB -> Tap "Rs. 250" -> Tap "Zomato" (recent chip) -> Tap "Save" (Large sticky bottom button). Total time: < 3 seconds.

5.4 Budget Screen (Weekly / Monthly / Yearly)

- Weekly Horizon: Shows a 7-day bar chart (Monday to Sunday) mapping daily expenses against the pro-rated weekly budget limit. Sat/Sun columns are highlighted.
- Monthly Horizon: Renders overall budget progress in a circular progress ring. Lists all active categories with progress bars colored using RAG formatting (Green: <75%, Amber: 75-99%, Red: >=100%). Budgets can be edited inline by tapping the amount.
- Yearly Horizon: A dual bar chart comparing Income vs. Expense by month. A toggle selector alternates between standard calendar years and the Indian Financial Year (April to March). Clicking any month's bar redirects the user to that month's monthly budget view.

5.5 Category Management Screen

- A drag-and-drop tree interface mapping Parent categories to their Subcategories.
- Displays a badge showing the current month's total spent in each category.
- Swipe actions allow archiving a category. A "Merge" button allows transferring all historical transactions from a source category to a target category before archiving.

6. Business Rules (BR)

- BR-001: Sub-5 Second Transaction Target: The Quick-Add interaction model must prioritize minimum taps. The UI must pre-select the last-used account and default the date to "Today."
- BR-002: Role Boundaries:
- Members cannot view other members' linked individual bank accounts (only transactions assigned to the household).

- Viewers are barred from reading `transactions.notes` or `attachments.storage\_key` to preserve user privacy.
- BR-003: Split Transaction Validation: The sum of values inside the `transaction\_splits` table must exactly equal `transactions.amount`. A database transaction must write both records atomically.
- BR-004: Custom Category Threshold: A single household is capped at a maximum of 100 custom categories to maintain UI responsiveness and indexing performance.
- BR-005: Default Currency Fallback: All `budget\_lines` planned values and savings goal targets are assumed to be denominated in the parent `households.currency`. Any transactions recorded in a foreign currency must store the transaction amount in its original currency and record `exchange\_rate` at the date of input.
- BR-006: Audit Log Integrity: Any updates to transactions, budgets, or accounts must record an audit entry. The application code must intercept write payloads and nullify values mapped to sensitive properties (`password\_hash`, `totp\_secret`, `session\_token`) before logging.
- BR-007: Idempotency Key Enforcements: The client must attach a unique UUID v4 as `idempotency\_key` on all `POST` requests. The API server must store these keys in Redis for 24 hours to prevent duplicate writes during network retries.

7. Database Requirements & Schema (DDL)

```
USERS ||--o{ HOUSEHOLD_MEMBERS : "belongs to"
HOUSEHOLDS ||--o{ HOUSEHOLD_MEMBERS : "has"
HOUSEHOLDS ||--o{ ACCOUNTS : "owns"
HOUSEHOLDS ||--o{ BUDGETS : "has"
HOUSEHOLDS ||--o{ SAVINGS_GOALS : "has"
HOUSEHOLDS ||--o{ CATEGORIES : "custom"
HOUSEHOLDS ||--o{ BILLS : "tracks"
HOUSEHOLDS ||--o{ LOANS : "manages"
HOUSEHOLDS ||--o{ INVESTMENTS : "holds"
USERS ||--o{ TRANSACTIONS : "creates"
ACCOUNTS ||--o{ TRANSACTIONS : "records"
CATEGORIES ||--o{ TRANSACTIONS : "classifies"
TRANSACTIONS ||--o{ TRANSACTION_SPLITS : "split into"
TRANSACTIONS ||--o{ ATTACHMENTS : "has"
TRANSACTIONS ||--o{ TRANSACTION_TAGS : "tagged with"
TAGS ||--o{ TRANSACTION_TAGS : "applied via"
BUDGETS ||--o{ BUDGET_LINES : "contains"
CATEGORIES ||--o{ BUDGET_LINES : "limits"
SAVINGS_GOALS ||--o{ GOAL_CONTRIBUTIONS : "receives"
RECURRING_RULES ||--o{ TRANSACTIONS : "generates"
USERS ||--o{ NOTIFICATIONS : "receives"
LOANS ||--o{ LOAN_PAYMENTS : "has"
BILLS ||--o{ BILL_PAYMENTS : "has"
TRANSACTIONS ||--o{ LOAN_PAYMENTS : "pays"
TRANSACTIONS ||--o{ BILL_PAYMENTS : "pays"
```

The database must use PostgreSQL 16. The schema must be built using Prisma ORM.

```
-- Core Identity & Access Control
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```



```
email VARCHAR(320) UNIQUE NOT NULL,
password_hash VARCHAR(255) NULL, -- Nullable for OAuth users
full_name VARCHAR(255) NOT NULL,
avatar_url VARCHAR(500) NULL,
timezone VARCHAR(64) NOT NULL DEFAULT 'Asia/Kolkata',
locale VARCHAR(10) NOT NULL DEFAULT 'en-IN',
currency CHAR(3) NOT NULL DEFAULT 'INR',
is_email_verified BOOLEAN NOT NULL DEFAULT FALSE,
is_2fa_enabled BOOLEAN NOT NULL DEFAULT FALSE,
totp_secret VARCHAR(255) NULL, -- Encrypted at application layer
last_login_at TIMESTAMPTZ NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
deleted_at TIMESTAMPTZ NULL
);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_deleted ON users(deleted_at) WHERE deleted_at IS NULL;

CREATE TABLE households (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) NOT NULL,
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  timezone VARCHAR(64) NOT NULL DEFAULT 'Asia/Kolkata',
  plan VARCHAR(32) NOT NULL DEFAULT 'free', -- free, premium, family
  created_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE household_members (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role VARCHAR(32) NOT NULL, -- owner, co_owner, member, viewer
  display_name VARCHAR(100) NULL,
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  invited_by UUID REFERENCES users(id),
  CONSTRAINT uq_household_user UNIQUE (household_id, user_id)
);
CREATE INDEX idx_members_user ON household_members(user_id);

-- Accounts & Categorization
CREATE TABLE accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  type VARCHAR(32) NOT NULL, -- checking, savings, credit_card, loan, investment, cash, other
  institution VARCHAR(255) NULL,
  account_number_last4 CHAR(4) NULL,
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  opening_balance DECIMAL(18,2) NOT NULL DEFAULT 0.00,
  current_balance DECIMAL(18,2) NOT NULL DEFAULT 0.00,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
```

```
color CHAR(7) NULL, -- Hex color
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_accounts_household ON accounts(household_id);

CREATE TABLE categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID REFERENCES households(id) ON DELETE CASCADE, -- Null means system seeded category
  parent_id UUID REFERENCES categories(id) ON DELETE SET NULL,
  name VARCHAR(100) NOT NULL,
  type VARCHAR(32) NOT NULL, -- income, expense, transfer
  icon VARCHAR(64) NULL,
  color CHAR(7) NULL,
  is_system BOOLEAN NOT NULL DEFAULT FALSE,
  is_archived BOOLEAN NOT NULL DEFAULT FALSE,
  sort_order INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_categories_household ON categories(household_id);
CREATE INDEX idx_categories_parent ON categories(parent_id);

-- Transactions & Splits
CREATE TABLE transactions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  account_id UUID NOT NULL REFERENCES accounts(id) ON DELETE RESTRICT,
  created_by UUID NOT NULL REFERENCES users(id),
  category_id UUID REFERENCES categories(id) ON DELETE SET NULL, -- Null if split transaction
  type VARCHAR(32) NOT NULL, -- income, expense, transfer
  amount DECIMAL(18,2) NOT NULL,
  currency CHAR(3) NOT NULL DEFAULT 'INR',
  exchange_rate DECIMAL(12,6) NOT NULL DEFAULT 1.000000,
  date DATE NOT NULL,
  description VARCHAR(500) NULL,
  notes TEXT NULL,
  is_recurring BOOLEAN NOT NULL DEFAULT FALSE,
  recurring_id UUID NULL, -- Links to scheduler rule if needed
  transfer_to_account_id UUID REFERENCES accounts(id) ON DELETE SET NULL,
  is_reimbursable BOOLEAN NOT NULL DEFAULT FALSE,
  reimbursed_at DATE NULL,
  is_reviewed BOOLEAN NOT NULL DEFAULT FALSE,
  loan_id UUID NULL, -- Backlink populated if paying loan
  bill_id UUID NULL, -- Backlink populated if paying bill
  investment_id UUID NULL, -- Backlink populated if purchasing investment
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  deleted_at TIMESTAMPTZ NULL
);
CREATE INDEX idx_transactions_household ON transactions(household_id);
CREATE INDEX idx_transactions_date ON transactions(date);
CREATE INDEX idx_transactions_composite ON transactions(household_id, date) WHERE deleted_at IS NULL;
CREATE INDEX idx_transactions_loan ON transactions(loan_id) WHERE loan_id IS NOT NULL;
```

```
CREATE INDEX idx_transactions_bill ON transactions(bill_id) WHERE bill_id IS NOT NULL;
CREATE INDEX idx_transactions_investment ON transactions(investment_id) WHERE investment_id IS NOT NULL;

CREATE TABLE transaction_splits (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  category_id UUID NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
  amount DECIMAL(18,2) NOT NULL,
  notes VARCHAR(255) NULL
);
CREATE INDEX idx_splits_transaction ON transaction_splits(transaction_id);

CREATE TABLE tags (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  name VARCHAR(50) NOT NULL,
  color CHAR(7) NULL,
  CONSTRAINT uq_household_tag UNIQUE (household_id, name)
);

CREATE TABLE transaction_tags (
  transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  tag_id UUID NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
  PRIMARY KEY (transaction_id, tag_id)
);

CREATE TABLE attachments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  file_name VARCHAR(255) NOT NULL,
  file_type VARCHAR(50) NOT NULL,
  file_size INT NOT NULL,
  storage_key VARCHAR(500) NOT NULL,
  thumbnail_key VARCHAR(500) NULL,
  uploaded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  uploaded_by UUID NOT NULL REFERENCES users(id)
);

-- Budgets & Goals
CREATE TABLE budgets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  month DATE NOT NULL, -- Seeded to first day of month (e.g. 2026-06-01)
  total_income_budget DECIMAL(18,2) NULL, -- In household currency
  notes TEXT NULL,
  created_by UUID NOT NULL REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT uq_household_budget_month UNIQUE (household_id, month)
);

CREATE TABLE budget_lines (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
budget_id UUID NOT NULL REFERENCES budgets(id) ON DELETE CASCADE,
category_id UUID NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
planned_amount DECIMAL(18,2) NOT NULL, -- In household currency
rollover_amount DECIMAL(18,2) NOT NULL DEFAULT 0.00,
is_percentage BOOLEAN NOT NULL DEFAULT FALSE,
percentage_value DECIMAL(5,2) NULL,
alert_at_pct INT NOT NULL DEFAULT 80,
weekly_override DECIMAL(18,2) NULL
);
CREATE INDEX idx_budget_lines_budget ON budget_lines(budget_id);

CREATE TABLE savings_goals (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  target_amount DECIMAL(18,2) NOT NULL,
  current_amount DECIMAL(18,2) NOT NULL DEFAULT 0.00,
  deadline DATE NULL,
  icon VARCHAR(64) NULL,
  color CHAR(7) NULL,
  linked_account_id UUID REFERENCES accounts(id) ON DELETE SET NULL,
  status VARCHAR(32) NOT NULL DEFAULT 'active', -- active, paused, completed, abandoned
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE goal_contributions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  goal_id UUID NOT NULL REFERENCES savings_goals(id) ON DELETE CASCADE,
  transaction_id UUID REFERENCES transactions(id) ON DELETE SET NULL,
  amount DECIMAL(18,2) NOT NULL,
  contributed_by UUID NOT NULL REFERENCES users(id),
  contributed_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  notes VARCHAR(255) NULL
);

-- Advanced Financial Entities
CREATE TABLE loans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  lender VARCHAR(255) NOT NULL,
  loan_type VARCHAR(32) NOT NULL, -- home, auto, personal, student, credit_card, other
  principal_amount DECIMAL(18,2) NOT NULL,
  outstanding_balance DECIMAL(18,2) NOT NULL,
  interest_rate DECIMAL(6,4) NOT NULL,
  emi_amount DECIMAL(18,2) NULL,
  emi_day SMALLINT NULL,
  start_date DATE NOT NULL,
  end_date DATE NULL,
  account_id UUID REFERENCES accounts(id) ON DELETE SET NULL
);

CREATE TABLE loan_payments (
```

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
loan_id UUID NOT NULL REFERENCES loans(id) ON DELETE CASCADE,
transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
payment_date DATE NOT NULL,
principal_amount DECIMAL(18,2) NOT NULL,
interest_amount DECIMAL(18,2) NOT NULL,
extra_payment DECIMAL(18,2) NOT NULL DEFAULT 0.00
);
CREATE INDEX idx_loan_payments_loan ON loan_payments(loan_id);
CREATE INDEX idx_loan_payments_txn ON loan_payments(transaction_id);

CREATE TABLE investments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  type VARCHAR(32) NOT NULL, -- stock, mutual_fund, etf, fd, ppf, nps, crypto, real_estate, other
  units DECIMAL(18,6) NULL,
  buy_price DECIMAL(18,2) NULL,
  current_price DECIMAL(18,2) NULL,
  current_value DECIMAL(18,2) NOT NULL,
  invested_amount DECIMAL(18,2) NOT NULL,
  account_id UUID REFERENCES accounts(id) ON DELETE SET NULL,
  last_updated TIMESTAMPTZ NOT NULL,
  notes TEXT NULL
);

CREATE TABLE bills (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,
  name VARCHAR(255) NOT NULL,
  category_id UUID NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,
  amount DECIMAL(18,2) NOT NULL,
  is_variable BOOLEAN NOT NULL DEFAULT FALSE,
  due_day SMALLINT NOT NULL,
  remind_days_before SMALLINT NOT NULL DEFAULT 3,
  account_id UUID REFERENCES accounts(id) ON DELETE SET NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE bill_payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  bill_id UUID NOT NULL REFERENCES bills(id) ON DELETE CASCADE,
  transaction_id UUID NOT NULL REFERENCES transactions(id) ON DELETE CASCADE,
  billing_period DATE NOT NULL, -- Date format e.g. 2026-06-01 representing month
  paid_at TIMESTAMPTZ NOT NULL,
  amount_paid DECIMAL(18,2) NOT NULL,
  CONSTRAINT uq_bill_period UNIQUE (bill_id, billing_period)
);
CREATE INDEX idx_bill_payments_bill ON bill_payments(bill_id);
CREATE INDEX idx_bill_payments_txn ON bill_payments(transaction_id);

-- System Configurations & Monitoring
```

```
CREATE TABLE recurring_rules (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,  
  template_data JSONB NOT NULL,  
  frequency VARCHAR(32) NOT NULL, -- daily, weekly, biweekly, monthly, quarterly, annually  
  start_date DATE NOT NULL,  
  end_date DATE NULL,  
  next_occurrence DATE NOT NULL,  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  created_by UUID NOT NULL REFERENCES users(id)  
);  
  
CREATE TABLE notifications (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  household_id UUID REFERENCES households(id) ON DELETE SET NULL,  
  type VARCHAR(64) NOT NULL,  
  title VARCHAR(255) NOT NULL,  
  body TEXT NOT NULL,  
  action_url VARCHAR(500) NULL,  
  is_read BOOLEAN NOT NULL DEFAULT FALSE,  
  sent_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  read_at TIMESTAMPTZ NULL  
);  
CREATE INDEX idx_notifications_user_read ON notifications(user_id, is_read);  
  
CREATE TABLE audit_log (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id),  
  household_id UUID NULL REFERENCES households(id) ON DELETE SET NULL,  
  entity_type VARCHAR(64) NOT NULL,  
  entity_id UUID NOT NULL,  
  action VARCHAR(16) NOT NULL, -- CREATE, UPDATE, DELETE  
  before_data JSONB NULL, -- Sensitive fields redacted in app layer  
  after_data JSONB NULL, -- Sensitive fields redacted in app layer  
  ip_address INET NULL,  
  user_agent VARCHAR(500) NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
-- Pre-aggregated Reporting Table  
CREATE TABLE monthly_snapshots (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  household_id UUID NOT NULL REFERENCES households(id) ON DELETE CASCADE,  
  month DATE NOT NULL,  
  category_id UUID NOT NULL REFERENCES categories(id) ON DELETE RESTRICT,  
  total_spent DECIMAL(18,2) NOT NULL DEFAULT 0.00,  
  total_income DECIMAL(18,2) NOT NULL DEFAULT 0.00,  
  budgeted_amount DECIMAL(18,2) NOT NULL DEFAULT 0.00,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT uq_household_month_cat UNIQUE (household_id, month, category_id)  
);
```

```
CREATE INDEX idx_snapshots_lookup ON monthly_snapshots(household_id, month);
```

8. API Requirements (REST Endpoints)

All request and response objects must conform to standard JSON structures.

8.1 Authentication

- `POST /api/v1/auth/register` (Register a new user)
- `POST /api/v1/auth/login` (Issue access token + set HttpOnly cookie refresh token)
- `POST /api/v1/auth/logout` (Revoke active session tokens)

Sample Login Request

```
{
  "email": "sharma.arjun@gmail.com",
  "password": "Password@123"
}
```

Sample Login Response (200 OK)

```
{
  "status": "success",
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "token_type": "Bearer",
    "expires_in": 900,
    "user": {
      "id": "c85d852a-cf91-4475-b827-2c974c865181",
      "full_name": "Arjun Sharma",
      "email": "sharma.arjun@gmail.com",
      "avatar_url": "https://lh3.googleusercontent.com/a/ALm5wu..."
    }
  }
}
```

8.2 Transactions

- `GET /api/v1/households/:hid/transactions` (Search and filter transactions)
- `POST /api/v1/households/:hid/transactions` (Log a new transaction)
- `PATCH /api/v1/households/:hid/transactions/:id` (Update fields)
- `DELETE /api/v1/households/:hid/transactions/:id` (Soft delete)

Sample Log Transaction Request

```
{
  "account_id": "7fa12bfb-cf98-468e-bf89-8d7ef2bc1234",
}
```

```
"category_id": "b328a6f2-cf12-421f-82ff-89dffab04321",
"type": "expense",
"amount": 350.00,
"currency": "INR",
"date": "2026-06-20",
"description": "Swiggy Dinner Order",
"tags": ["food", "delivery"],
"idempotency_key": "4d16c5b0-379e-4b47-b359-5ad12fc33be4"
}
```

Sample Response (201 Created)

```
{
  "status": "success",
  "data": {
    "id": "8fa21ef2-cf98-468e-bf89-8d7ef2bc8273",
    "type": "expense",
    "amount": 350.00,
    "description": "Swiggy Dinner Order",
    "category": {
      "id": "b328a6f2-cf12-421f-82ff-89dffab04321",
      "name": "Food Delivery"
    },
    "date": "2026-06-20",
    "created_at": "2026-06-20T21:15:00Z"
  }
}
```

8.3 Budgets & Summaries

- `GET /api/v1/households/:hid/budgets/summary` (Retrieve Weekly / Monthly / Yearly budget visual data)

Sample Request URL

```
`GET
/api/v1/households/3ca12bcf-cf89/budgets/summary?period_type=yearly&reference_date=2026-06-20&year_type=financial`
```

Sample Response (200 OK)

```
{
  "status": "success",
  "data": {
    "period": "yearly",
    "year_type": "financial",
    "reference_date": "2026-06-20",
    "summary": {
      "total_budgeted": 960000.00,
      "total_actual": 482340.00,
      "projected_total": 890000.00
    },
    "categories": [
```

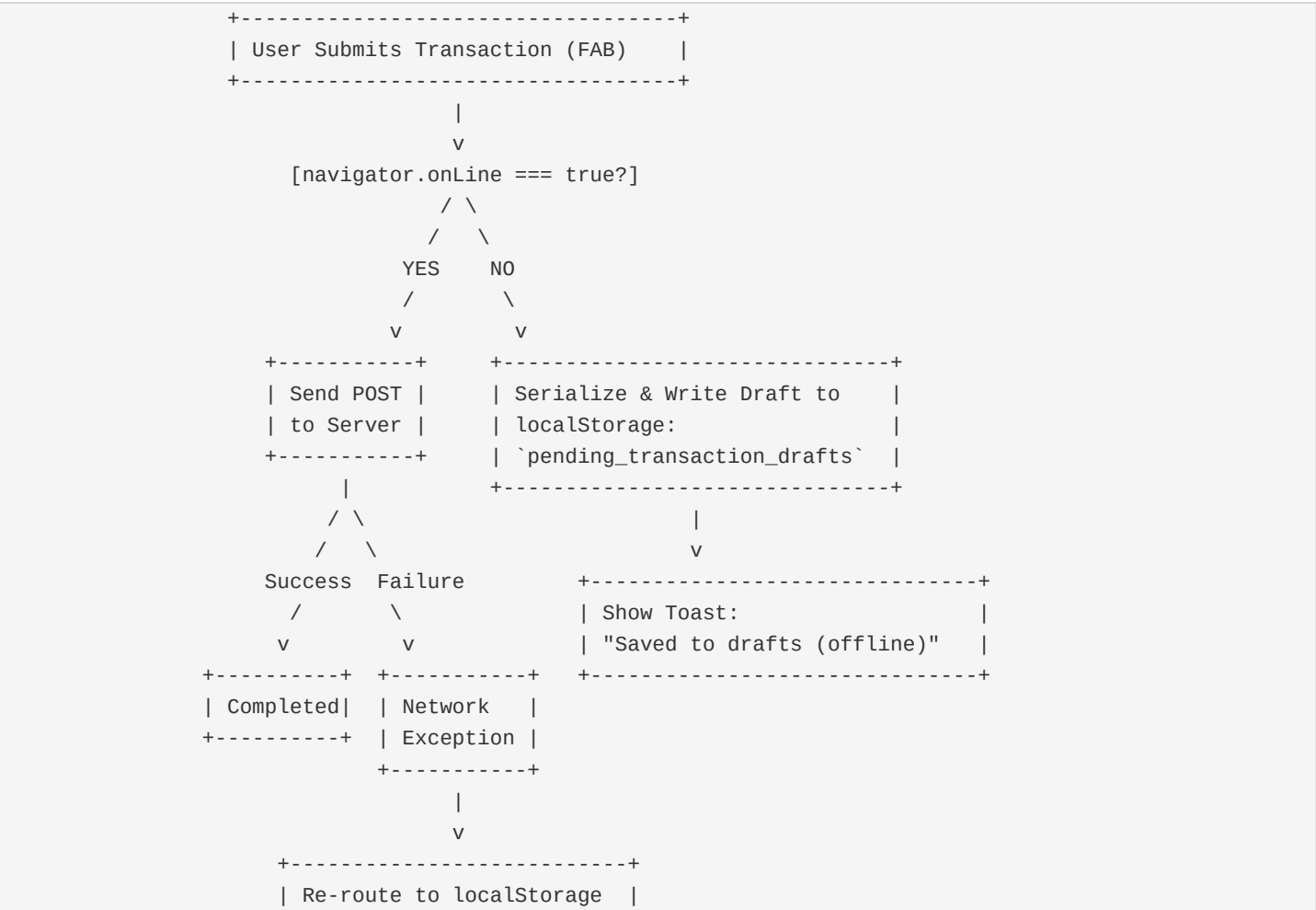


```
{
  "id": "cat-food-uuid",
  "name": "Food & Dining",
  "annual_budget": 72000.00,
  "ytd_actual": 41200.00,
  "projected_full_year": 82400.00,
  "status": "over_projected"
}
]
```

9. Error Handling & Connectivity (Offline Drafts)

Mobile devices often navigate through dead zones (elevators, basement parking, offline shops). The system must handle network dropouts gracefully without corrupting remote state.

9.1 The Local Draft Queue Workflow



draft list
+-----+

9.2 Draft Sync Protocol

- Queue Ingestion: When a transaction fails to submit due to an offline state or server timeout, the client intercepts the state, vibrates the device for 50ms using ``navigator.vibrate(50)``, saves the item to the ``pending_transaction_drafts`` array in ``localStorage``, and displays a toast message: `""Saved to local drafts (offline)""`.
- Sync Trigger: The application registers event listeners on ``window.addEventListener('online', triggerSync)``.
- Sync UI banner: Upon detecting an online state, a banner is rendered at the top of the dashboard page: `""You have X offline transaction drafts. [Sync Now] | [Review]""`.
- Resolution: Tapping Sync Now initiates a batch submission. The client loops through drafts, posting them using the stored ``idempotency_key``. Successful transfers are popped from the local array. Any validation errors halt the loop, displaying the affected draft inside the review modal.

10. Security & Privacy Specs

- SEC-001: Encryption Key Management: Database field-level encryption must rely on an environment-managed variable ``ENCRYPTION_SECRET`` (32-byte key). Columns must utilize ``AES-256-GCM`` formatting, saving initialization vectors (IV) alongside ciphertext.
- SEC-002: Token Refresh Mechanism: Access tokens expire in 15 minutes. Refresh tokens are stored in an `HttpOnly`, `Secure`, `SameSite=Strict` cookie with a path of ``/api/v1/auth/refresh``. Accessing ``/refresh`` invalidates the old token pair and issues rotated credentials.
- SEC-003: Password Hashing: Passwords must be hashed using ``bcrypt`` with a cost factor of 12. Password criteria are validated on both client-side and server-side forms.
- SEC-004: Audit Log Sanitization: The application's Prisma query engine client must register middleware to block sensitive fields. The fields ``password_hash``, ``totp_secret``, ``session_token``, and credit card values must be stripped from metadata schemas during create or update logs.

11. Testing & Quality Assurance

The application targets a comprehensive testing suite to assure functionality across target viewports.

11.1 Test Matrix (Target Devices)

Tests must run locally and in CI/CD pipeline using emulated viewports:

Emulated Device	Width	Height	OS	Key Targets
---	---	---	---	---

Samsung Galaxy A-series	360px	800px	Android Chrome	RAG bar layouts, FAB positioning
Xiaomi Redmi Note	393px	851px	Android Chrome	Scroll container limits (no horizontal scroll)
iPhone SE (3rd Gen)	375px	667px	iOS Safari	Safe-area notch and home bar heights
iPhone 14 Pro	393px	852px	iOS Safari	Dynamic island padding compliance

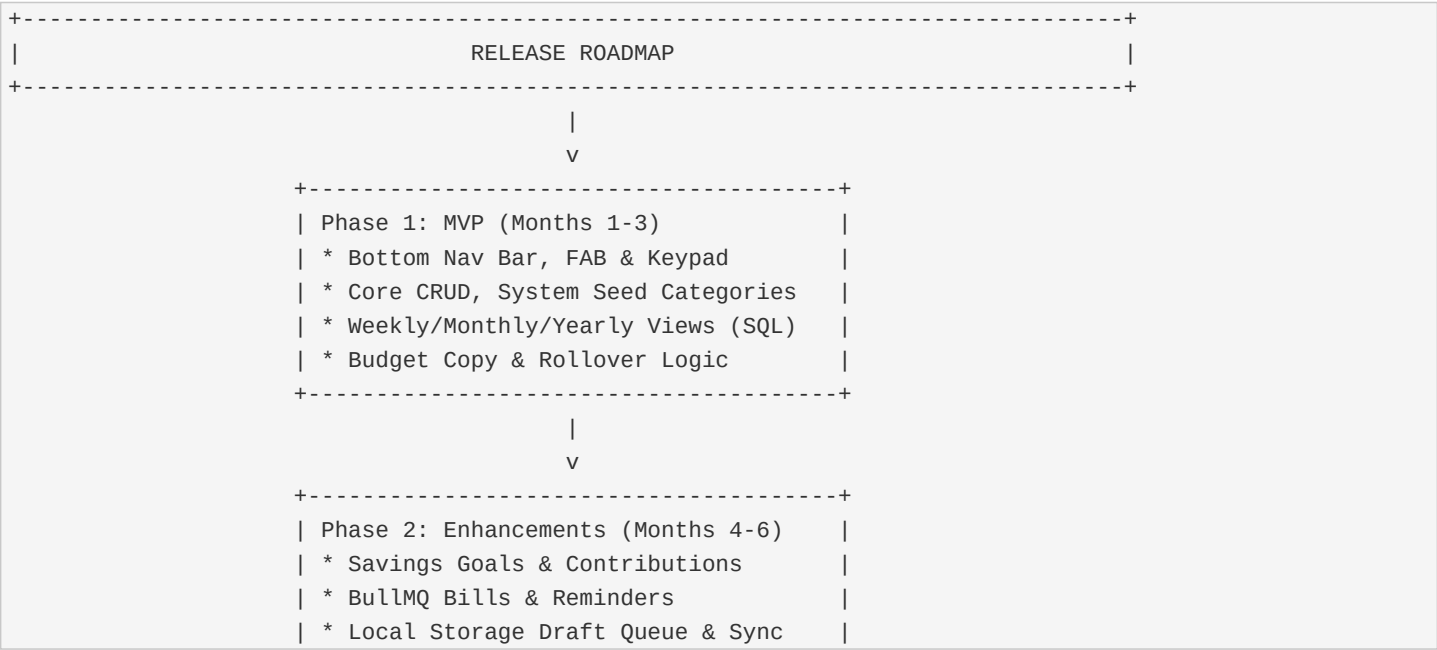
11.2 End-to-End Automated Testing (Playwright)

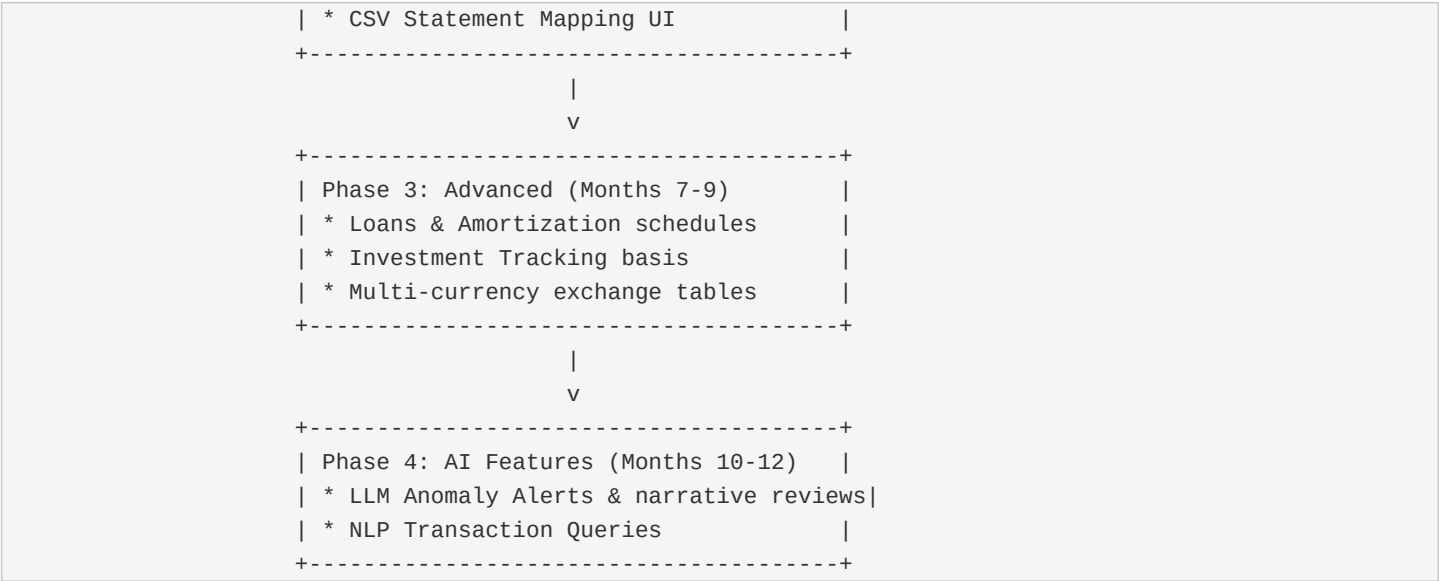
- Playwright E2E tests must validate the following paths:
- User registration -> Onboarding -> Create household -> Seed categories (Verify 23 initial categories appear).
 - Add transaction (under 5-second entry target) -> verify remaining budget progress ring calculates correctly -> verify colors shift based on limits.
 - Set network offline -> attempt transaction entry -> verify draft is captured in `localStorage` -> toggle network online -> sync pending draft -> verify server database consistency.

11.3 Performance Benchmarks (k6)

- Dashboard requests must resolve in < 2.0 seconds with 100 concurrent virtual users.
- Batch inserts of 100 transactions must execute in < 3.0 seconds under standard DB operations.

12. Release Phases & Scope





12.1 Phase 1 (MVP) Scope

- Core Authentication: Email registration, login, and Google OAuth flow.
- Core Household: Create/Join households and assign Roles.
- Categories: 23 system seeded categories and Custom Category CRUD.
- Visualizer: Weekly, Monthly, and Yearly budget views with dynamic switches.
- Transactions: Standard CRUD entry via custom calculator keyboard.
- Aggregations: Database snapshot cron generating monthly records.

12.2 Phase 2 Scope (Deferred)

- Savings Goals tracking and contributions ledger.
- BullMQ bills processing engine and email notifications.
- CSV Import mapper with duplicate validation algorithms.
- Local draft sync queue.
- PWA manifests.

12.3 Phase 3 Scope (Advanced)

- Loans tracking and principal vs interest amortizations.
- Investment assets (mutual funds, cryptos) cost basis.
- Split transactions database constraints.

12.4 Phase 4 Scope (AI Features)

- Fine-tuned text categorization model based on description.
- Outlier detection comparing current transaction with historic averages.
- Narrative builder querying Gemini Flash API generating text monthly stories.